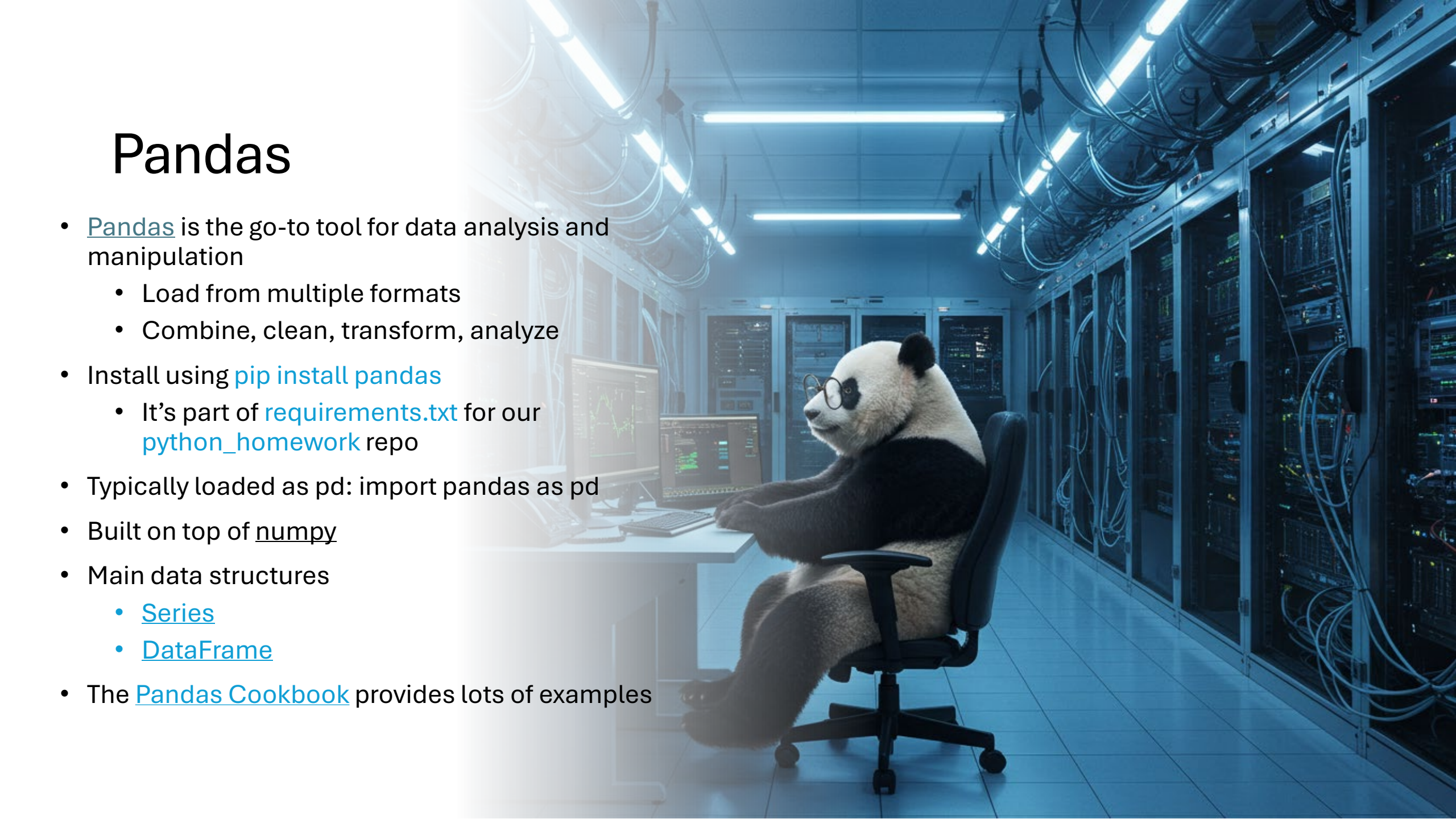
CTD Python Essentials

Week 4:

Introduction to Data Engineering

Pandas

- Pandas is the go-to tool for data analysis and manipulation
  - Load from multiple formats
  - Combine, clean, transform, analyze
- Install using pip install pandas
  - It's part of requirements.txt for our python_homework repo
- Typically loaded as pd: import pandas as pd
- Built on top of numpy
- Main data structures
  - Series
  - DataFrame
- The Pandas Cookbook provides lots of examples



pd.Series

- One-dimensional array with axis labels (including time series).
  - Stored in a 1 dimensional numpy ndarray
- Customizable indices
  - Any hashable object
  - Default to sequential integers
  - Don’t need to be unique
  - my_series[‘string-index’]
  - Can also access by integer position – iloc[<int>]
- Constructor - pandas.Series(data=None, index=None, name=None)
  - data: Array-like, iterable, dict or scalar, index taken from dict keys unless specified
  - index: Sequence of hashable values, same length as data
  - name: Can be named

pd.DataFrame

- Two-dimensional, size-mutable, potentially heterogeneous tabular data.
- Customizable row index (any hashable type)
- Customizable column index (any hashable type)
- Also accessible sequentially using iloc
- Constructor - pandas.DataFrame(data=None, index=None, columns=None)
  - data: ndarray, Iterable, dict, or DataFrame – lots of flexibility
  - index: index (labels) for rows – sequential integers if unspecified/not in data
  - columns: column labels – sequential integers if unspecified/not in data
- The copy() method for DataFrame and Series provides a deep copy by default.

Initializing DataFrames

- A few examples, lots of options

```python
# Data: list of lists
data = [
    [1, 'Alice', True],
    [2, 'Bob', False],
    [3, 'Charlie', True]
]

# Column names
columns = ['ID', 'Name', 'Active']

# Create DataFrame
df_list_of_lists = pd.DataFrame(data, columns=columns)
```

```python
# Data: list of dicts
data = [
    {'ID': 1, 'Name': 'Alice', 'Age': 24},
    {'ID': 2, 'Name': 'Bob', 'Age': 30},
    {'ID': 3, 'Name': 'Charlie', 'Age': 29}
]

# Create DataFrame
df_list_of_dicts = pd.DataFrame(data)
```

```python
# Data: dict of lists
data = {
    'Country': ['USA', 'Canada', 'Mexico'],
    'Capital': ['Washington', 'Ottawa', 'Mexico City'],
    'Population_Millions': [331, 38, 128]
}

# Create DataFrame
df_dict_of_lists = pd.DataFrame(data)
```

```python
# Data: dict of pandas Series
data = {
    'A': pd.Series([10, 20, 30], index=[0, 1, 2]),
    'B': pd.Series([100, 200, 300], index=[0, 1, 2])
}

# Create DataFrame
df_dict_of_series = pd.DataFrame(data)
```

Initializing DataFrames - Missing or misaligned data

- Handles missing data flexibly

```python
# Data: dict of pandas Series
data = {
    'A': pd.Series([10, 20, 30], index=[0, 1, 2]),
    'B': pd.Series([100, 200], index=[0, 1])    # Missing index 2
}

# Create DataFrame
df_dict_of_series = pd.DataFrame(data)

print(df_dict_of_series)
```

```
    A      B
0  10  100.0
1  20  200.0
2  30    NaN
```

CSV and JSON input/output

- DataFrames can be loaded from or written to a variety of standard file formats.
- CSV – Comma Separate Values
  - df = pd.read_csv('data.csv')
  - DataFrame.to_csv(filepath, sep=',', index=True, header=True, encoding=None)
  - Can specify a separator other than comma
- JSON – JavaScript Object Notation
  - df = pd.read_json('data.json')
  - DataFrame.to_json(filepath) # lots of formatting options
- Excel spreadsheets
  - pd.read_excel("path_to_file.xls", sheet_name="Sheet1")
  - Uses the openpyxl package from pypi

Data Selection

- Use the loc method to access rows and columns via labels
  - view = df.loc[7, 'City'] # 7 is a label which happens to be an int
- Use the iloc method to access rows and columns via integer position
  - view = df.iloc[5, 7] # positional from top left
- Row first, then column for loc and iloc
- Selecting columns
  - df['height'] # returns a Series
  - df[['height', 'width']] # returns a new DataFrame with just the selected columns
  - df['col']['index']
    - Not recommended – the second index is an index into the selected Series and may not be unique. It's intentionally not referred to as a row here.
    - Use loc or iloc for 2d selection
- Conditional selection
  - view = df[df['Level'] > 9000]
- Creates a view into the DataFrame which should be treated as read-only

Combining DataFrames

- Using the concat() method
  - combined_df = pd.concat([df1, df2], ignore_index=True)
    - Creates new sequential indices
- Using the merge() method
  - Useful for shared keys
  - merged = pd.merge(df1, df2, on='ID')
- Using the join() method
  - Index based
  - df1 = pd.DataFrame({'Age': [25, 30]}, index=['Alice', 'Bob'])
  - df2 = pd.DataFrame({'Salary': [70000, 80000]}, index=['Alice', 'Bob'])
  - joined = df1.join(df2)

Introduction to Data Cleaning

- Introduced in lesson 4, covered in more detail in later lessons
- replace()
- to_numeric()
- fillna() replace NA and NaN values
- ffill() and bfill()
- Series.str methods
  - strip()
  - upper()
  - lower()
- to_datetime()

Data Inspection

- The head(n) and tail(n) methods provide a nicely formatted quick look at a few rows (5 default).
- The info() method summarizes a variety of DataFrame details
  - Index dtype and columns, non-null values and memory usage
- The mean() method can be used to find the average of a column (or a given axis)
- The nunique() method counts unique values for an axis.
- There are Series and DataFrame versions
- There are lots of analysis methods

Q&A and Demo

Github gist with code examples:
Intro to Pandas